



MANUAL TÉCNICO

LECTOR PDF MINIMALISTA

ELABORADO POR

Samuel Zúñiga Vega
Rubén Hurtado Granados
Andrés Alexander Feng Wu
Victor Gabriel Mejías Salas
Xotchil Daniela Rojas Fernández
Fabián Araya Ortega

SAN JOSÉ
2026

RESUMEN

Este es el manual técnico para el proyecto de Lector PDF Minimalista. Su propósito es familiarizar al desarrollador con el proyecto y su diseño. Este proyecto nace de la necesidad de un lector de documentos pdf que no sea intrusivo y no sature la vista. Esta necesidad fue expresada por el cliente Rodolfo Mora, él es quien recibe el producto final. Para solucionar este problema el objetivo general del proyecto es desarrollar una aplicación de escritorio para Windows que permita visualizar y modificar documentos pdf mediante una interfaz limpia, minimalista y clara.

La aplicación tiene funcionalidades de modificación de pdf (eliminar, reordenar, concatenar páginas, etc), visualización de pdf, presentación de pdf, entre otras. La implementación utiliza el framework Electron junto con bibliotecas especializadas para el manejo de pdf (PDF.js, pdf-lib). La arquitectura de la aplicación es modular, se separan las funciones en módulos independientes. El manual cuenta con instrucciones para descargar el código y desarrollar, para compilar y para desplegar en el entorno final. Al final, se comparte el aprendizaje acumulado junto con las problemáticas y limitaciones de la aplicación.

Palabras clave: Lector PDF, estilo minimalista.

Índice general

1. Introducción	1
1.1. Sobre este manual	1
1.2. Descripción y Alcance del proyecto	1
1.2.1. Antecedentes	1
1.2.2. Objetivos	2
1.2.3. Contacto	2
2. Especificación de funcionalidad	3
2.1. Descripción del producto	3
2.1.1. Historias de usuario	3
3. Arquitectura del sistema	8
3.1. Diseño general del sistema	8
3.1.1. Diagrama de dependencias	8
3.1.2. Diagrama de bloques	9
3.1.3. Diagrama de Clases	10
4. Instrucciones de Instalación	12
4.1. Entorno de Desarrollo	12
4.1.1. Dependencias	12
4.1.2. Código fuente	12
4.2. Entorno de Producción	13
4.2.1. Instrucciones de compilación	13
4.2.2. Dependencias	13
4.2.3. Instrucciones de instalación	13
5. Conclusiones y trabajo futuro	14
5.1. Conclusiones	14
5.2. Problemáticas y limitaciones	14
5.3. Trabajo futuro	14

Índice de figuras

3.1. Diagrama de Dependencias: proceso principal	9
3.2. Diagrama de Dependencias: proceso renderer	9
3.3. Diagrama de Bloques	10
3.4. Diagrama de Clases	11

Capítulo 1

Introducción

1.1. Sobre este manual

Este manual está dirigido a nuevos desarrolladores que quieren trabajar en este proyecto. Su objetivo es familiarizar a los nuevos desarrolladores con el proyecto, y brindar tanto las herramientas como la información necesaria para que puedan desarrollar sobre el proyecto de manera rápida y efectiva.

Este manual contiene el contexto del proyecto y los objetivos que quiere alcanzar (Capítulo 1), así como las funcionalidades necesarias para alcanzar dichos objetivos (Capítulo 2). También, se incluye la información sobre cómo se diseñó el sistema, y cómo están implementadas las funcionalidades requeridas (Capítulo 3). Además, contiene guías para configurar el entorno de desarrollo y para desplegar el sistema (Capítulo 4). Finalmente, se incluyen reflexiones útiles sobre el proceso de desarrollo, las limitaciones del proyecto y las oportunidades de mejora que existen (Capítulo 5).

1.2. Descripción y Alcance del proyecto

1.2.1. Antecedentes

Este proyecto surge como respuesta a una necesidad directa expresada por el cliente, quien en la actualidad usa distintos lectores de PDF y manifiesta una insatisfacción con estas herramientas debido a sus interfaces sobrecargadas de utilidades, barras de herramientas y menús poco intuitivos. Además, también menciona experiencias negativas con lectores alternativos, que son más livianos, pero tienen algunos problemas de estabilidad, entre otras cosas.

Como referencia de estilo visual el cliente resaltó la interfaz que utiliza un lector desarrollado por Augmented Text Company (<https://www.augmentedtext.info/reader>), que corresponde a un visor sin barra de título, sin bordes intrusivos y con un estilo minimalista blanco sobre blanco agradable a la vista.

Beneficiario

El beneficiario directo del proyecto es el cliente Rodolfo Mora, quien en su labor académica maneja de forma constante documentos en formato PDF (lecturas, presentaciones exportadas y documentos legales que requieren firma), y que actualmente no cuenta con una herramienta que combine una interfaz minimalista con las funcionalidades específicas que necesita.

Importancia

La importancia del proyecto radica en ofrecer una alternativa amigable a las aplicaciones de lectura y edición de PDF existentes en el mercado que, según lo expresado por el cliente, priorizan la

cantidad de funcionalidades visibles por encima de la experiencia de lectura sobrecargando la interfaz y dificultando la concentración en el contenido.

Grupos o comunidades beneficiadas

El principal beneficiado es el cliente. Sin embargo, dado que se contempla como un proyecto de software libre, cualquier usuario de Windows (y eventualmente Linux) que busque un lector y editor de PDF minimalista podrá beneficiarse de la herramienta desarrollada.

Necesidades solventadas

Interfaces sobrecargadas y poco intuitivas de los lectores de PDF actualmente disponibles (Adobe Acrobat, Foxit, entre otros), que dificultan una experiencia de lectura centrada en el contenido.

1.2.2. Objetivos

■ Objetivo General

- Desarrollar una aplicación de escritorio para Windows que permita visualizar y modificar documentos PDF mediante una interfaz minimalista, limpia y clara.

■ Objetivos Específicos

- Construir un visualizador de documentos pdf que muestre las páginas del documento al usuario.
- Desarrollar funcionalidades para hacer modificaciones simples al pdf, como eliminar, compaginar y mezclar páginas, y que se guarden en el mismo archivo o uno nuevo.
- Implementar la búsqueda, selección y copiado de texto en un pdf que se está visualizando.
- Desarrollar un modo presentación para el pdf que se está visualizando.
- Diseñar una interfaz minimalista con color blanco que muestre las funcionalidades de la aplicación solo cuando sea necesario, intentando mantener poca información en la pantalla para no saturar al usuario.

1.2.3. Contacto

Para consultas y soporte, contacte a: samuelzv18@gmail.com

Capítulo 2

Especificación de funcionalidad

2.1. Descripción del producto

El producto es una aplicación de escritorio para el sistema operativo Windows. La aplicación funciona como un lector de documentos PDF, que también cuenta con funciones para hacer modificaciones. Sigue una filosofía minimalista que se refleja en la UX/UI del programa.

El lector tiene funcionalidades básicas como zoom, scroll, búsqueda de texto, salto a una página específica y selección de texto con posibilidad de copiar. Además, renderiza las páginas aplicando Lazy Loading para mejorar el rendimiento. También, tiene una sección para mostrar el outline del documento y otra para mostrar la lista de paginas del documento. Por último, el lector tiene modo de presentación en pantalla completa.

La aplicación también tiene un módulo de herramientas que modifican y editan el archivo pdf. Es posible eliminar páginas y concatenar páginas de otro archivo pdf. También se incluyen las funciones de guardar y guardar como, para conservar las modificaciones en los documentos.

La codificación para las historias de usuario es US-XXX, donde X representa el número consecutivo de historias de usuario.

Durante el desarrollo se realizan más tareas que utilizan otro tipo de codificación. Estas tareas no se incluyen en este documento porque son de tipo administrativo o de mantenimiento y no son relevantes para entender el producto final (así que tampoco son relevantes entre sprints).

Para las tareas sobre documentación del código se usa la codificación: DC-XXX. Para las tareas de refactorización del código se usa la codificación: FX-XXX.

2.1.1. Historias de usuario

Lector PDF minimalista

Código	Descripción	Criterios de Calidad
US-001	App minimalista: Ocultar la información en la interfaz de usuario para dar un estilo minimalista a la aplicación. Esto se hace mediante la implementación de estilos de css que modifican la interfaz del renderer y configuraciones de la ventana de electron en el proceso principal..	Como usuario, quiero una interfaz limpia y minimalista para la aplicación, de modo que pueda enfocarme en la lectura del contenido sin distracciones visuales..

Código	Descripción	Criterios de Calidad
US-002	Compaginar archivos pdf: Compaginar 2 archivos pdf abiertos para obtener un archivo pdf resultante que es la unión del final del primer pdf con el inicio del segundo pdf. Esto se hace utilizando la biblioteca pdf-lib (en el proceso principal).	Como usuario, quiero poder compaginar múltiples documentos o páginas, para organizar la estructura del documento final de manera lógica antes de exportarlo.
US-005	Visualización ajustada al PDF: Las dimensiones de la ventana se ajustan al visualizador del documento pdf. Esto se logra usando funciones de electron para modificar la ventana, y scripts en el renderer que comunican el tamaño del visualizador.	Como usuario, quiero que la aplicación renderice y adapte correctamente la visualización del documento al tamaño de mi ventana, para una experiencia ergonómica.
US-006	Presentar PDF: Un pdf abierto puede mostrarse en modo presentación. Para esto se abre una nueva ventana de electron en modo pantalla completa. Esta ventana carga un visualizador diferente en el renderer que implementa las funcionalidades del modo presentación. El visualizador está implementado como una clase de javascript que internamente utiliza la biblioteca PDF.js	Como usuario, quiero activar un modo de presentación a pantalla completa, para poder exponer el contenido del documento a una audiencia ocultando los menús y herramientas.
US-007	Eliminar páginas de un PDF: Eliminar páginas de un documento pdf cargado en memoria. Se elige una lista con cualquiera de las páginas del documento para ser eliminadas. Internamente se utiliza la biblioteca pdf-lib para modificar el archivo (en el proceso principal).	Como usuario, quiero poder seleccionar y eliminar páginas específicas de un PDF, para remover información irrelevante o incorrecta del documento original.
US-008	Guardar un PDF: Un pdf abierto, cargado en memoria, y modificado se puede guardar en el disco en la misma ruta que estaba el archivo original (sobrescribiendo el archivo). Esto se hace usando las funciones de la biblioteca estándar de nodejs para escribir archivos .	Como usuario, quiero poder guardar los cambios realizados sobre el archivo PDF actual, para asegurar y no perder el progreso de mi edición.
US-009	US-009: Reordenar páginas de un PDF: La lista de páginas contiene un botón que permite reordenar las páginas arrastrándolas en la interfaz. Luego la información de la lista de páginas se transfiere al servicio de reordenamiento que utiliza pdf-lib internamente.	Como usuario, quiero poder reordenar las páginas del documento (por ejemplo, arrastrándolas), para ajustar el flujo de lectura y la organización según mis necesidades.

Código	Descripción	Criterios de Calidad
US-010	Outline de PDF: Desplegar un outline que contiene la estructura del documento pdf. Este outline es jerárquico. Es posible que algunos documentos pdf no tengan outline, en ese caso no se despliega un outline. Esta funcionalidad está implementada en el renderer mediante javascript, haciendo uso de la información que está en la sección del visualizador de pdf (US-015).	Como usuario, quiero visualizar el índice (outline) del documento en un panel lateral, para comprender rápidamente la jerarquía y estructura general del texto.
US-011	Navegar Outline: Los elementos de la sección del outline son clickeables, cuando el usuario presiona un título del outline el visualizador de pdfs renderiza la página que contiene ese título. Esto se hace mediante scripts de javascript en el renderer.	Como usuario, quiero poder hacer clic en cualquier sección del índice (outline) y saltar directamente a esa página, para navegar ágilmente a través de documentos muy extensos.
US-012	Guardar Nuevo PDF: El documento pdf abierto y cargado en memoria se puede guardar en el disco como un nuevo documento, independiente de la fuente original. Para obtener la ruta del nuevo documento se utiliza los dialogs que provee electron, y para escribir el archivo se usan las funciones de la biblioteca fs de Node.js.	Como usuario, quiero tener la opción de "Guardar como" para generar un nuevo archivo con mis modificaciones, manteniendo el PDF original intacto como respaldo. Con el fin de recuperarlo fácilmente si necesito usarlo.
US-013	Buscar Texto: Una combinación de teclas invoca un campo de texto donde se puede escribir la cadena de texto que se va a buscar dentro del documento pdf. Luego de escribir el visualizador del documento resalta el texto que coincide. Además del campo de texto, hay controles para navegar por el visualizador para encontrar todas las coincidencias. Esto se hace mediante scripts de javascript en el renderer que usan la información desplegada en el visualizador.	Como usuario, quiero utilizar una barra de búsqueda para encontrar palabras o frases específicas dentro del documento, para localizar datos puntuales sin tener que leer todo el texto.

Código	Descripción	Criterios de Calidad
US-014	Abrir un PDF: El usuario selecciona el documento pdf que quiere abrir. Puede ser mediante un dialog del sistema, internamente implementado con funciones de la biblioteca de electron. O, puede ser mediante el arrastre del archivo a la aplicación, internamente implementado con eventos del navegador en el renderer. Sin importar el método, al final el archivo es guardado como una variable global en el proceso principal para ser usado en cualquier parte del sistema.	Como usuario, quiero poder abrir un archivo PDF desde el explorador de archivos de mi dispositivo, para acceder a su contenido dentro de la aplicación.
US-015	Ver páginas de un PDF: Un pdf abierto se renderiza en el visualizador de la aplicación. Este trabajo lo hace la clase PdfViewer en base a la información del documento que está cargada en memoria. Este trabajo se hace en el renderer. Internamente se utiliza la biblioteca PDF.js.	Como usuario, quiero poder visualizar las páginas individuales del documento, para poder consumir la información que contienen.
US-016	Copiar texto portapapeles: Cuando se selecciona texto del visualizador (US-019) el usuario puede copiar el texto al portapapeles del sistema con un atajo del teclado. Esta funcionalidad es parte de la clase PdfViewer y proviene de la biblioteca PDF.js (ejecutado en el renderer).	Como usuario, quiero poder copiar el texto previamente seleccionado al portapapeles del sistema, para pegarlo y reutilizarlo en otras herramientas o correos.
US-017	Zoom de un PDF: Mediante atajos de teclado y el scroll del mouse el usuario puede cambiar el nivel de zoom del visualizador de pdf. Esta funcionalidad es parte de la clase PdfViewer y proviene de la biblioteca PDF.js (ejecutado en el renderer).	Como usuario, quiero poder acercar (zoom in) y alejar (zoom out) la vista de la página, para visualizar detalles pequeños o evaluar la distribución completa de la hoja.
US-018	Scroll de un PDF: Mediante el scroll del mouse el usuario puede navegar verticalmente por el documento renderizado por el visualizador. Esta función incluye lazy loading de las páginas visualizadas para mejorar el rendimiento del programa. Esta funcionalidad es parte de la clase PdfViewer y proviene de la biblioteca PDF.js (ejecutado en el renderer).	Como usuario, quiero poder desplazarme verticalmente (scroll) a lo largo del documento, para mantener una experiencia de lectura continua y fluida.
US-019	Seleccionar texto: El visualizador de pdf incluye una capa extra sobre el pdf renderizado que le permite al usuario seleccionar texto del documento. Esta funcionalidad es parte de la clase PdfViewer y proviene de la biblioteca PDF.js (ejecutado en el renderer).	Como usuario, quiero poder seleccionar bloques de texto arrastrando el cursor, como acción indispensable antes de poder copiar(US-016) o resaltar el contenido.

Código	Descripción	Criterios de Calidad
US-020	Saltar a Página: El usuario puede ingresar un número de página en el mismo elemento que se usa para desplegar la página actual. Al hacerlo, el visualizador cambia su posición vertical para que dicha página quede centrada en la pantalla y se renderice. Esta funcionalidad es parte de la clase PdfViewer y proviene de la biblioteca PDF.js (ejecutado en el renderer).	Como usuario, quiero poder ingresar un número específico de página, para saltar a esta inmediatamente sin tener que scrollar.
US-021	Mostrar Conteo de Páginas: Un elemento de la interfaz muestra la página actual que está renderizada en el visualizador y la cantidad total de páginas del documento. Esta funcionalidad es parte de la clase PdfViewer y proviene de la biblioteca PDF.js (ejecutado en el renderer).	Como usuario, quiero poder la página en la que estoy en relación a la cantidad total de páginas, para saber con certeza en que parte del documento me encuentro.
US-022	Menú Click Derecho: Al hacer click derecho en cualquier punto de la interfaz se abre un menú contextual que despliega botones para ejecutar las funcionalidades de la aplicación. El menú contextual es construido usando funciones de la biblioteca de electron y utiliza elementos de la interfaz del sistema. Las funciones que se activan en el menú son callbacks que se inyectan desde por medio la biblioteca IPC de electron.	Como usuario, quiero poder mostrar un menú con click derecho, para acceder a todas las funcionalidades del sistema sin que estas se entrometan en el lector.

Capítulo 3

Arquitectura del sistema

3.1. Diseño general del sistema

La aplicación está construida sobre el framework Electron, lo que influencia en gran medida la arquitectura del sistema. Una aplicación en Electron tiene un proceso de ejecución principal que corre Node.js, este se encarga de la comunicación con el sistema operativo, la creación de ventanas, entre otras cosas; utilizando las bibliotecas que provee Electron. El proceso principal a su vez puede crear uno o más procesos renderizadores (renderer process), cada uno con una ventana asignada, estos procesos corren un navegador Chromium que se encarga de desplegar la interfaz de la aplicación (aunque también puede realizar más tareas con scripts de javascript). Esto provoca que la arquitectura este dividida en 2 partes: el proceso principal y el renderer.

En esta aplicación el renderer es el encargado de desplegar la interfaz de usuario principal donde se visualizan los documentos pdf y los botones, y demás elementos, usados para las otras funcionalidades del sistema. El proceso principal se encarga de la creación de ventanas del sistema (como la ventana principal, diálogos del sistema, menú contextual, etc), la comunicación con el sistema operativo, las funcionalidades que modifican los documentos y cuenta con el estado de la aplicación.

El código del proyecto tiene un diseño modular basado en los módulos de javascript. En cada módulo se encuentran funciones exportadas (públicas) y las funciones no exportadas (privadas) que solo se utilizan a lo interno del módulo. Cada módulo es responsable de una funcionalidad del sistema. Usualmente un módulo del proceso principal tiene una versión análoga en el renderer.

Para comunicar el proceso principal con el proceso renderer Electron provee la biblioteca IPC (Inter-Process Communication). En el proyecto la biblioteca se utiliza en el módulo 'ipc' que trabaja como una capa de interfaz entre los servicios que provee el proceso principal y el proceso renderer. El proceso renderer se conecta con la interfaz del módulo ipc a través de un preload script que electron carga en el renderer cuando es invocado, este archivo se encuentra en la carpeta raíz del proyecto con el nombre preload.js.

3.1.1. Diagrama de dependencias

El diagrama de dependencias muestra las dependencias entre los módulos de la aplicación. Es una forma útil de visualizar como trabajan los módulos de la aplicación entre sí y de ver que forma se relacionan. Este diagrama es generado utilizando la herramienta madge, así que muestra el proyecto tal y como está programado. También, puede resultar útil para detectar dependencias circulares o un flujo extraño dentro del programa.

El formato del diagrama es el siguiente. Cada archivo o módulo del sistema es un rectángulo, el color representa el estado de la dependencia: azul si tiene dependencias, verde si no tiene dependencias y rojo si es parte de una dependencia circular. Las flechas indican las dependencias entre módulos.

Hay un diagrama que muestra las dependencias del proceso principal (Ver figura 3.1) y otro que muestra el proceso renderer (Ver figura 3.2).

Proceso principal

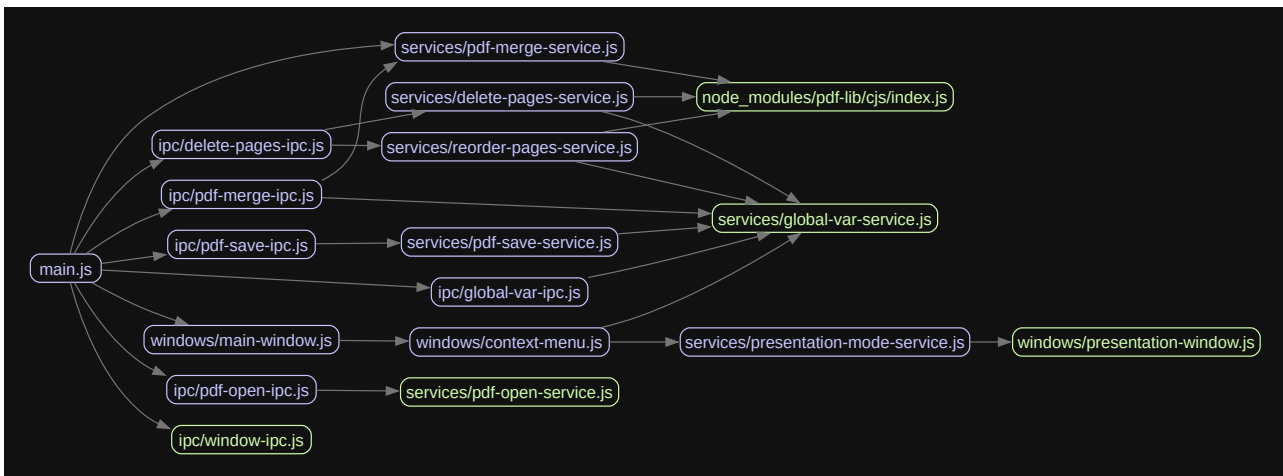


Figura 3.1: Diagrama de Dependencias: proceso principal

Proceso renderer

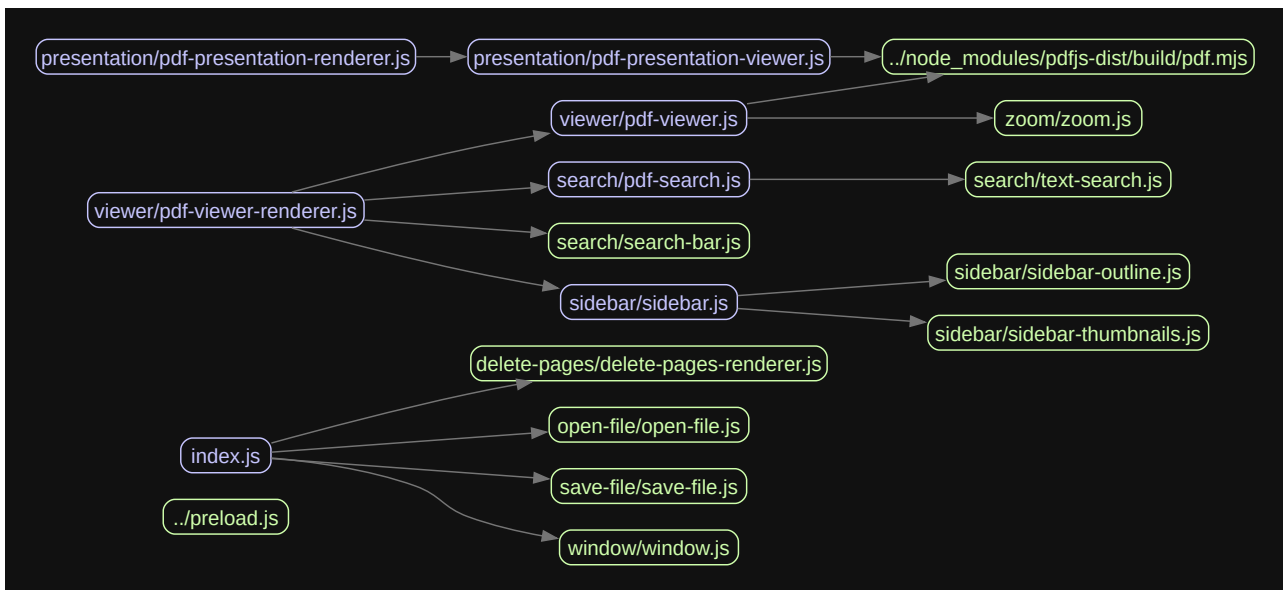


Figura 3.2: Diagrama de Dependencias: proceso renderer

3.1.2. Diagrama de bloques

El diagrama de bloques muestra la organización de los módulos del proyecto y cómo se comunican entre sí de forma superficial. Este diagrama es útil para entender la estructura del proyecto ya que la jerarquía de módulos es similar a la estructura de archivos que tiene el proyecto.

El diagrama (Ver figura 3.3) expone 2 bloques principales: proceso principal y renderer. El bloque del proceso principal tiene el módulo "windows" que tiene ventanas del sistema, el módulo "services" que tiene módulos con funcionalidades del programa y el módulo "ipc" que expone módulos del proceso principal al proceso renderer. El bloque del proceso renderer tiene 2 módulos: index y presentation, cada uno corresponde a una página de la interfaz de la aplicación.

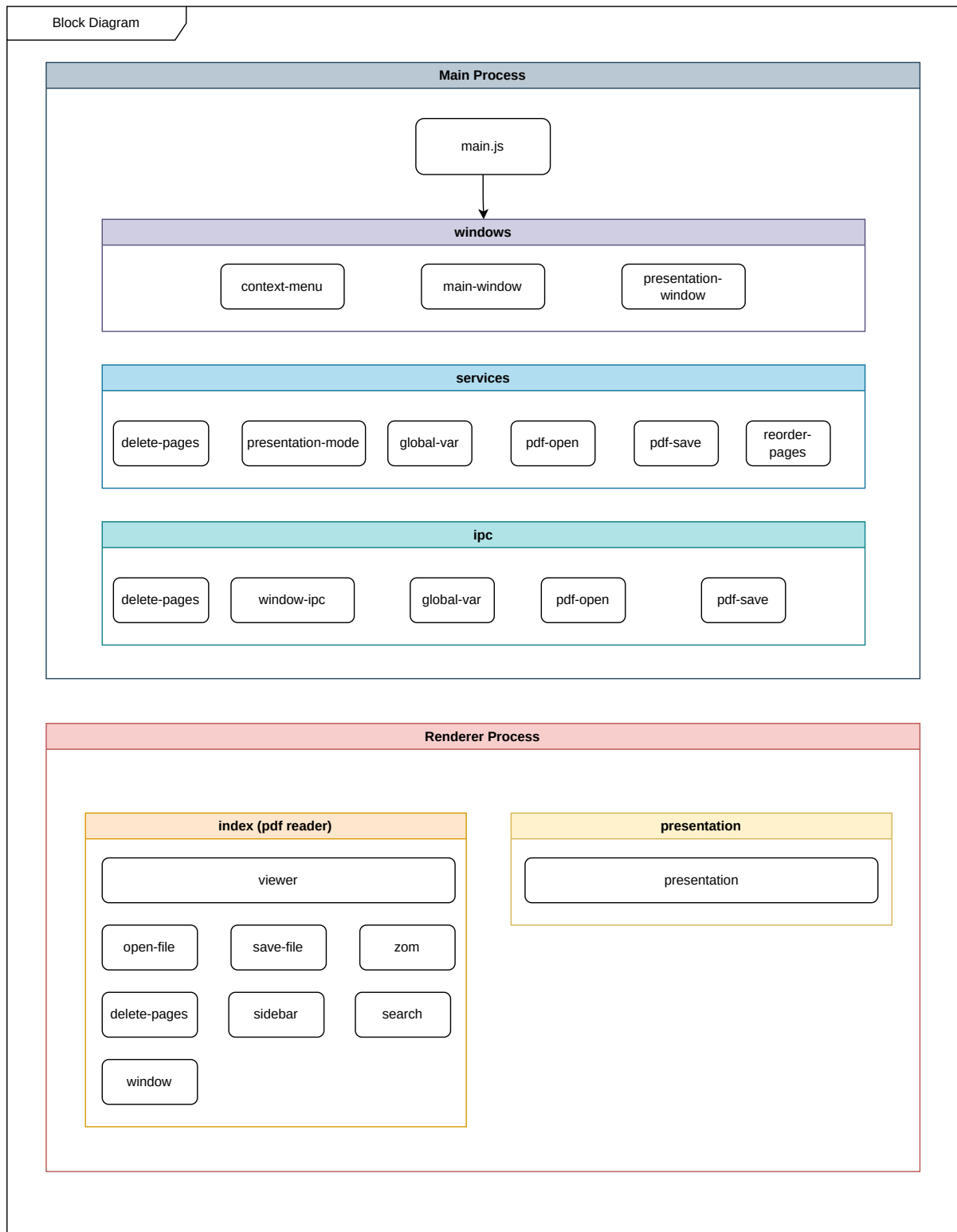


Figura 3.3: Diagrama de Bloques

3.1.3. Diagrama de Clases

El diagrama de clases describe las clases que existen dentro de la aplicación. El diagrama (Ver figura 3.4) muestra 2 clases. La clase PdfViewer es la encargada de mostrar el pdf dentro de la aplicación, además muestra información como número de página actual y tiene herramientas como

el zoom. La clase PdfPresentationViewer se encarga de mostrar el pdf en modo presentación.

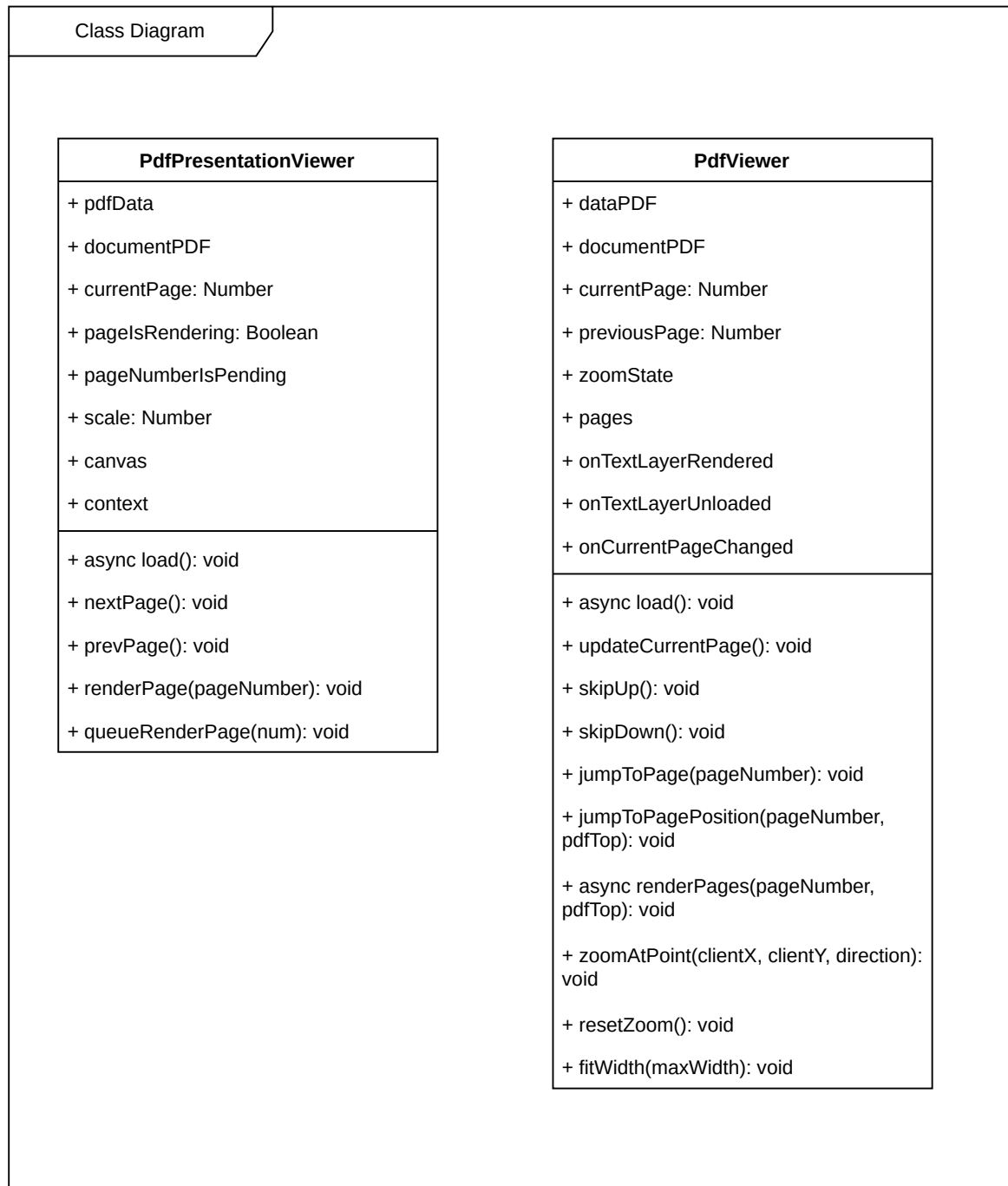


Figura 3.4: Diagrama de Clases

Capítulo 4

Instrucciones de Instalación

4.1. Entorno de Desarrollo

4.1.1. Dependencias

La aplicación está construida con el framework Electron que necesita el entorno de ejecución Node.js. Además, la aplicación utiliza bibliotecas de npm para algunas funcionalidades, así que también se debe descargar esa herramienta.

Lista de pasos para preparar el entorno de desarrollo:

1. **Github** El repositorio del proyecto se encuentra en Github. Para trabajar colaborativamente en el repositorio necesita registrarse en este servicio. <https://github.com/>.
2. **Git** El administrador de versiones del proyecto es Git. Para descargarlo siga las instrucciones en <https://git-scm.com/install/>.
3. **VSCode** El editor de texto sugerido para desarrollar el proyecto es VSCode. Para descargarlo siga las instrucciones en <https://code.visualstudio.com/download>.
4. **Node.js y NPM** Descargue la versión más reciente de Node.js siguiendo las instrucciones en <https://nodejs.org/es/download>.

4.1.2. Código fuente

Lista de pasos para descargar el código fuente y configurar lo necesario para empezar a trabajar:

1. **Acceder al repositorio** Pida a un administrador del proyecto que le conceda permisos para trabajar en el repositorio del proyecto.
2. **Descargar código fuente** Puede descargar el código fuente de varias maneras. El comando ‘git clone’ puede descargar el código del repositorio si git cuenta con los credenciales de Github. Para hacerlo puede seguir la guía en <https://docs.github.com/en/get-started/git-basics/caching-your-github-credentials-in-git>
3. **Descargar Electron y otras bibliotecas** Ejecute el comando ‘npm ci’ en la carpeta raíz del proyecto.
4. **Descargar formatter:** Descargue Prettier en su editor de texto para que el código siga el formato definido. VSCode cuenta una extensión que configura Prettier de forma automática. También, se recomienda instalar ESLint para detectar errores en el código, VSCode también tiene una extensión para esta herramienta.

5. **Estándar de código:** Antes de programar lea el documento en "docs/standard.md". Este documento tiene una lista de buenas prácticas que se deben seguir en el desarrollo de la aplicación.
6. **Ejecutar aplicación en entorno de desarrollo:** Para ejecutar la aplicación usando el código fuente utilice el comando 'npm start'.

4.2. Entorno de Producción

4.2.1. Instrucciones de compilación

Para compilar el proyecto se ejecuta el comando 'npm run build'. Este comando usa la biblioteca electron-builder para compilar y empaquetar el proyecto para las plataformas: Windows y Linux.

4.2.2. Dependencias

El proceso de compilación empaqueta todas las dependencias necesarias en el ejecutable, por lo tanto el entorno de producción solo necesita tener el binario.

4.2.3. Instrucciones de instalación

El ejecutable es portable, así que no requiere instalarse en el sistema. Para ejecutar la aplicación solo correr el ejecutable.

Capítulo 5

Conclusiones y trabajo futuro

5.1. Conclusiones

El proceso renderer no cuenta con una forma de persistir datos pesados en memoria entre páginas o entre refrescos. Ya que el proceso renderer ejecuta Chromium las formas de guardar datos son el localStorage o sessionStorage, pero no son eficientes y tienen límites pequeños (¡10MB). Por lo tanto, la solución a este problema es mantener la información persistente en el proceso principal y hacer que el proceso renderer la extraiga cuando lo necesita. Este método tiene la ventaja de que también permite compartir información entre varios procesos renderer (varias ventanas de la aplicación). Es importante resaltar que la comunicación entre procesos conlleva un overhead, sin embargo en este proyecto se procesan pocos datos(¡1GB) y la latencia(¡500ms) no es un aspecto importante.

5.2. Problemáticas y limitaciones

Problemáticas

El tiempo de arranque de la aplicación es una problemática presente. Usualmente tarda más de 2 segundos en aparecer la ventana en pantalla, esto sin estar cargando ningún pdf.

Limitaciones

Los requerimientos mínimos del sistema son los siguientes:

- **RAM:** 200MB. Esta es la memoria para el programa base, si se carga un pdf entonces se necesita al menos una cantidad de memoria igual al tamaño del archivo.
- **Almacenamiento:** 250 MB. Tamaño del ejecutable de la aplicación.
- **Procesamiento:** Un procesador Intel i8 serie 8000 o similar. Es probable que funcione con menos procesamiento pero no ha sido probado.
- **Sistema operativo:** Se compilan ejecutables para los sistemas operativos Windows 11 y cualquier distribución de Linux que soporte el formato AppImage.

5.3. Trabajo futuro

Se recomienda implementar un documento de estilo global para normalizar aspectos del estilo a lo largo de la aplicación. Esto mejora la estética de la aplicación y facilita el desarrollo.

Se puede implementar una interfaz web del sistema.

Se puede implementar gestión de múltiples firmas con bloqueo de concurrencia. (Investigar acerca de Docusign).